MSc. in Computing
CSC1142 Cloud Technologies

Section 1: Student Details

Student ID:	A00049618, A00048674
Student name:	Megha Kanojia, Shubin Li
Student email	megha.kanojia2@mail.dcu.ie, shubin.li2@mail.dcu.ie
Chosen major:	Data Analytics
Date of Submission	27-11-2025

Section 2: Datasets and Source Code

Grammy Winners and Nominees from 1965 to 2024 (updated by its author)	https://www.kaggle.com/datasets/johnpendenque/grammy-winners-and-nominees-from-1965-to-2024
Grammy Winners and Nominees from 1965 to 2024 (older version)	https://gitlab.computing.dcu.ie/cloud-technologies-pyspark-project/grammy_and_spotify_analysis/-/blob/main/CloudData/grammy_winners.csv
Spotify_1Million_Tracks	https://www.kaggle.com/datasets/amitanshjoshi/spotify-1million-tracks
Gitlab repo for source code	https://gitlab.computing.dcu.ie/cloud-technologies-pyspark-project/grammy_and_spotify_analysis/-/blob/main/CloudTechProject.ipynb?plain=1
Google Colab link for complete source code **(please use colab link to view the first interactive graph)**	CloudTechProject.ipynb

Please Note: The Grammy Winners dataset has been updated by its author, the previous version of dataset on which we have worked has been added to our Gitlab repository.

Section 3: Video Demonstration

Please find the video explanation here: CloudTech Project.mp4

Section 4: Introduction and motivation for the pipeline and the choice of the technologies

We have created an end-to-end data engineering and analytics workflow to investigate the divergence between critical acclaim (Grammy Winners) and commercial success (Spotify

Popularity) in the music industry from 2000 to 2023. The pipeline produces raw data into three distinct cohorts for comparison:

1. Critically Acclaimed: "Record of the Year" Grammy winners.
2. Listener Hits: The top 100 most popular songs per year on Spotify.
3. General Population: A baseline of the general Spotify dataset to serve as a control group.

- We have built an ETL and EDA pipeline that ingests two messy datasets (the Grammy winners and the Spotify tracks data)
- Then we have cleaned and normalized the text fields.
- There were issues while parsing the datasets because of embedded quotes and multiline fields which we found during the pre-processing phase so we dealt with it and used special parsing techniques.
- Then we mapped the records across datasets using normalized text and manual index mapping (for a small section which did not map via normalized text) of artist name and their song or album name.
- After mapping, we cleaned the data further and ensured that we don't have missing values for features that we wanted to use.
- We extracted yearly top-N Spotify tracks, merged with Grammy dataset (common songs) and produced Parquet outputs for downstream analysis and visualization using Pandas, Matplotlib and Seaborn.

Technology used	**Primary role**	**Key Strength**	**Reason for choice**
Apache Spark / PySpark	Distributed ETL, joins, windowing functions	Scales to large datasets; efficient joins	Needed for large Spotify dataset and complex joins
Google Colab	Development environment	Fast prototyping; easy to run notebooks	Convenient for interactive development and Drive mounting
Google Drive (I/O)	Storage for CSV and Parquet (could be used later in any other analysis)	Simple persistence and sharing	Easy integration with Colab for small/medium projects; can store datasets with size in GBs
Parquet	Final storage format	Columnar, compressed, schema-preserving	Efficient for analytics and downstream reads
Pandas + Matplotlib/Seaborn	Visualization and EDA	Familiar plotting and interactivity	Good for exploratory plots after Spark pre-processing

Section 5: Related work

Existing systems and studies in this area mainly focus on music-genre visualization, music-trend prediction, and cross-platform comparisons based on audio features.

1. Music-genre visualization platforms

Every Noise at Once is a music discovery website created by former Spotify employee Glenn McDonald. It presents a directory of musical genres, artists, and playlist by Spotify, presented in a scatter plot word map style. It also provides feature-based tools for exploring similar songs.

Limitation: the platform focuses on streaming popularity and genre clustering but does not compare streaming popularity with professional evaluations such as the Grammy Awards.

2. Academic research

A number of papers focus on music-trend prediction and cross-platform comparisons using audio features, such as Spotify Data Analysis and Song Popularity Prediction and Exploring Trends in Music Platforms: A Comparative Analysis of Key Factors for Trending Songs on Spotify and YouTube.

Difference from our work: our project focuses on long-term descriptive analysis and the comparison of different groups, rather than building predictive models.

3. Public music data ETL projects

Several open-source projects, such as the “spotify-data-engineering-project” on GitHub, provide ETL pipelines for processing Spotify data. However, these projects typically perform only basic data transformation.

Difference from our work: our system integrates both Spotify and Grammy datasets and normalized the string column mismatches caused by textual noise.

a. Contributions of Our Approach

Our project offers the following features and advantages. First, we integrate multiple data sources by combining Spotify popularity and audio-feature data with Grammy awards data, creating a unified dataset that supports direct comparison across groups. During preprocessing, we address the challenge of matching records across datasets, which multiple columns contain noisy text. Through cleaning, splitting, and standardization, we have achieved precise mapping between artists and track titles. Based on matched data, we constructed an annually consistent dataset to support long-term trend analysis. Additionally, we developed interactive visualization tools that allow users to freely switch between features and statistical methods, providing clear and intuitive comparisons of differences across various groups.

b. References

[1] McDonald, G. (n.d.). Every Noise at Once. https://everynoise.com/

[2] Bethapudi, P. (2024, April 13). Spotify Data Analysis and Song Popularity Prediction. SSRN. http://dx.doi.org/10.2139/ssrn.4793176

[3] Charchyan, A. Exploring trends in music platforms: A comparative analysis of key factors for trending songs on Spotify and YouTube https://cse.aua.am/files/2024/05/Exploring-Trends-in-Music-Platforms-A-Comparative-Analysis-of-Key-Factors-for-Trending-Songs-on-Spotify-and-YouTube.pdf

[4] Srivastava, A. (n.d.). spotify- data- engineering- project [GitHub repository]. GitHub. https://github.com/ArpiteshSrivastava/spotify-data-engineering-project

Section 6: Description of the dataset

a. Source of the data:

The Grammy Awards Dataset (grammy_winners.csv)	This dataset contains records of award winners and nominees. For this specific pipeline, the focus is restricted to the **"Record of the Year"** category within the timeframe of **2000–2023**.	link
The Spotify Audio Features Dataset (spotify_data.csv)	This dataset is a large-scale repository of tracks containing songs data and their quantitative audio features.	link

1. grammy_winners.csv:
    a. Total records: 25,370
    b. Key Attributes: grammy_year, category, artist, song_or_album
    c. This dataset has been sourced from grammy.com and it compiles the historical Grammy Award winners and nominees, including details such as the award category, artist, song or album, and year of recognition. It can be used for music trend analysis, industry research, and historical insights.

*Note: This dataset has now been updated by its author on Kaggle, however we are using the older version in our project (link to this dataset used is provided in Section 2).

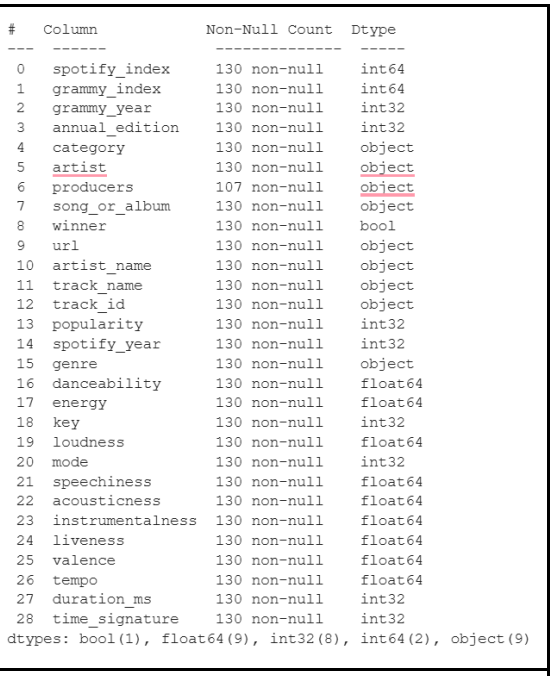
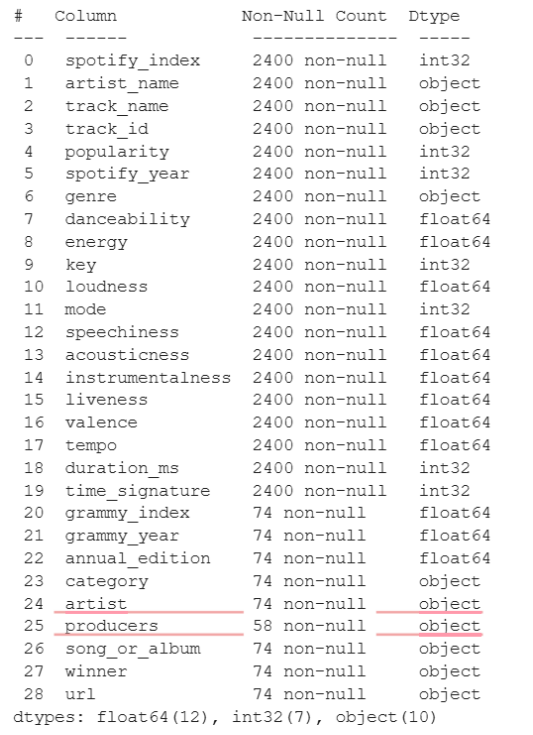
2. spotify_data.csv:
    a. Total records: 1,159,764
    b. Key Attributes: artist_name, track_name, spotify_year, popularity, genre, danceability, energy, loudness, speechiness, acousticness, valence, tempo
    c. This dataset was extracted from the Spotify platform using the Python library "Spotipy", which allows users to access music data provided via APIs.

Initial data schema and shape:

grammy data : grammy_winners.csv	shape:(25370, 9) # Column Non-Null Count Dtype --- ------ -------------- ----- 0 grammy_index 25370 non-null int64 1 grammy_year 25370 non-null int64 2 annual_edition 25370 non-null int64 3 category 25370 non-null object 4 artist 22644 non-null object 5 producers 3039 non-null object 6 song_or_album 25348 non-null object 7 winner 25370 non-null bool 8 url 25370 non-null object dtypes: bool(1), int64(3), object(5)
spotify data: spotify_data.csv	shape:(1159764, 20) # Column Non-Null Count Dtype --- ------ -------------- ----- 0 spotify_index 1159764 non-null int64 1 artist_name 1159749 non-null object 2 track_name 1159763 non-null object 3 track_id 1159764 non-null object 4 popularity 1159764 non-null int64 5 spotify_year 1159764 non-null int64 6 genre 1159764 non-null object 7 danceability 1159764 non-null float64 8 energy 1159764 non-null float64 9 key 1159764 non-null int64 10 loudness 1159764 non-null float64 11 mode 1159764 non-null int64 12 speechiness 1159764 non-null float64 13 acousticness 1159764 non-null float64 14 instrumentalness 1159764 non-null float64 15 liveness 1159764 non-null float64 16 valence 1159764 non-null float64 17 tempo 1159764 non-null float64 18 duration_ms 1159764 non-null int64 19 time_signature 1159764 non-null int64 dtypes: float64(9), int64(7), object(4)

Final data schema and shape:

grammy_parquet :grammy_group_parquet	Shape: (130, 29)

	
spotify_parquet: spotify_group_parquet	Shape:(2400, 29) 

b. Process of extraction/collection:

- The initial datasets were downloaded from Kaggle which were pre-processed and transformed into final analytical datasets through a robust ETL pipeline using Apache Spark.
- As per information available on Kaggle, provided by dataset's authors:
  1. The Grammy dataset was sourced from grammy.com

2. The Spotify dataset was extracted from Spotify platform using Spotipy Python library which uses APIs to access data.

Section 7: Description of data processing

Phase 1: Preparing our Grammy group

1. Loading our Grammy dataset as “grammy” and Spotify dataset as spotify
2. Check the number of rows present in our grammy dataset: 25,370
3. Check the number of rows present in our spotify dataset: 1,159,764
4. Check the columns and structure of spotify dataset
5. Check the structure of grammy to understand the features and their types
6. Grouping the grammy dataset by category column and filtering top 10 categories to select the one on which we will work in our pipeline.
7. Filtering the dataset for years from 2000 to 2023 only so that it matches with Spotify’s timeline
8. Choosing “Record of the Year” as our category and filtering dataset with this category.
9. Checking the missing value for song_or_album column: 0 found
10. Counting the number of winner counts for each year: approx. 5 until 2017
11. Dealing with messy data of song_or_album column, by using regex to remove whitespaces and brackets along with feat. Information inside brackets
12. Normalizing the text of artist and song_or_album columns by lowering their case so that there is no issue while merging them with Spotify data. We normalize the artist_name and track_name column of Spotify dataset in the similar manner.
13. Perform inner merge on these grammy and spotify datasets using artist and song_or_album with artist_name and track_name columns.
14. Now these are the results that matched properly (count = 90), but we noticed that there were still a few of them which were having same song or album and artist but did not match because of noise in them (count = 54). So we manually curated a dictionary of their indexes and tried to map them (found 41 out of 54 that matched).
15. Finally, after a vertical merge of matched and unmatched (manually matched) datasets we got our first dataset.
16. We checked for duplicates in this curated dataset and handled them by dropping the duplicate row (only 1 was found duplicate).
17. We saved this first curated dataset as grammy_group_parquet.

Phase 2: Preparing our Spotify group

1. First, we read the data from csv file using default commands.
2. We checked the structure of dataset columns and if they contain null values
3. Tried to filter the dataset using distinct spotify_year, but received messy results containing not just year but random text.
4. Investigated on this issue and found that the data is leaking from other columns which contain quotation mark and multilines.

    - spotify_year column is of string type

- track_name splits into more columns because we did not escape quotation marks
- Example 1:
  - 8559 actual track name: Don Giovanni, ossia Il dissoluto punito, K.527 / Act 2: "Don Giovanni, a cenar teco m'invitasti"
  - In pyspark trackname: "Don Giovanni, ossia Il dissoluto punito, K.527 / Act 2: ""Don Giovanni
  - In pyspark track_id : a cenar teco m'invitasti"""
- Example 2:
  - 8636 actual track name: All-night Vigil, Op. 37, "Vespers": Hail, O Virgin Mother (Ave Maria)
  - In pyspark trackname: "All-night Vigil, Op. 37, ""Vespers"": Hail
  - In pyspark track_id: O Virgin Mother (Ave Maria)"

Which implies:

- original content: ABC: "DEF,GH "
- in csv : "ABC: ""DEF,GH""" (quotation marks are creating issue in proper parsing of data and splitting them).

```
+----------------------+
|spotify_year          |
+----------------------+
|007VTPx3UOezmy7xWsrquW|
|6zgWk7VmYihSrU1UDrSThC|
|4ir1OyDVCLBFXePjhiIFEK|
|7                     |
|2QRL3aEqAyKCc88yfoEq3g|
|51                    |
|1QrtVtu28d9LsGQD56sqL3|
|3uZBqptJfTzUaF6ZVP27Ge|
|5MlzjVzTnolUIah7beG87r|
| Floßhilde)"          |
|1aLs8G7GNBWLloglhNP0Nd|
|15                    |
|2jwG0ipR11UNSI9XUzusQ0|
|54                    |
|2016                  |
|0orJChfDosI9HpzPVomcVS|
|48QL3WyaTFo6864EUiGiRQ|
|2012                  |
|2020                  |
|11                    |
+----------------------+
only showing top 20 rows
```

5. Hence, to fix this issue we use special options while reading our csv file:

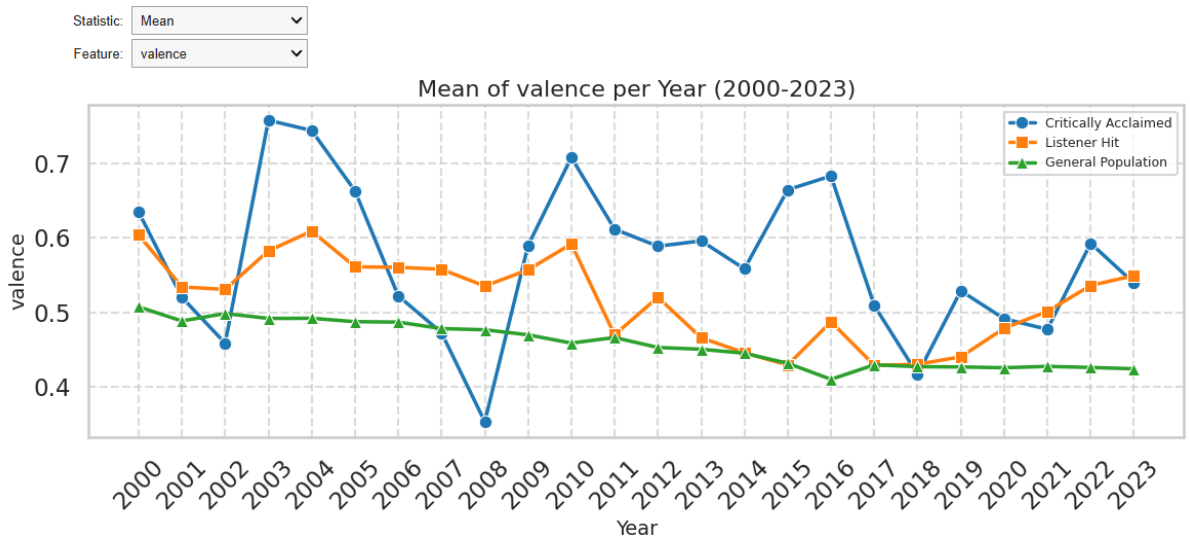
   .option("quote", "").option("escape", "").option("multiLine", True)

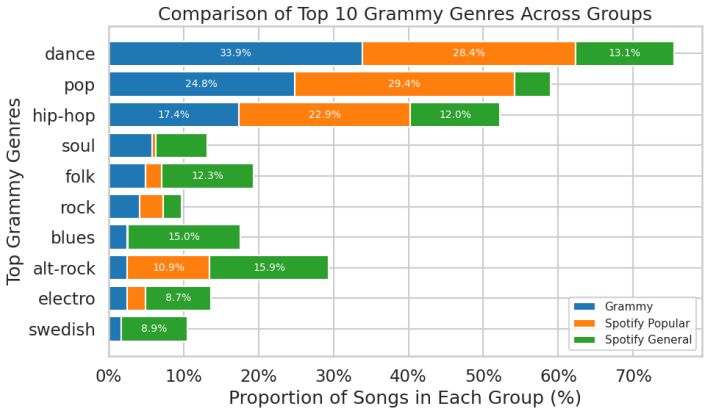
6. Now, we check distinct years in our dataset (found 2000 to 2023, as expected).
7. We partition the dataset by spotify_year, sort by popularity and extracted top 100 records for each year.
8. We then read the grammy group initially saved as parquet in drive.

9. We choose only the columns that belonged originally to Grammy dataset.
10. Then horizontally merge this grammy group with our filtered spotify records, so that our top 100 records for each year (total 2400 spotify records) have grammy features as well (null if they are not present in grammy group).
11. Only 74 records got mapped with grammy group, others have null in grammy features.
12. Save this dataset as spotify_group_parquet

Phase 3: Create visualizations using Pandas

1. Read all three datasets:
    a. spotify_popular_group – our Spotify group
    b. grammy_group – our Grammy group
    c. spotify_general_group – Spotify original data
2. Check their shape and columns
3. Select common audio features:
   'popularity', 'danceability', 'energy', 'key', 'loudness', 'mode', 'speechiness', 'acousticness', 'instrumentalness', 'liveness', 'valence', 'tempo', 'duration_ms','time_signature'
4. Calculate mean and median for these features grouped by year for each dataset
5. Finally draw a line plot showing mean and median of all audio features for each group across the timeline (2000-2023)
6. We can interact with the line plot's different audio parameters and mean or median values.
7. Plot a horizontal bar graph for the comparison of Top 10 Grammy Genres Across Groups.





Section 8: Development of the pipeline

In our pipeline we are leveraging the distributed computing for heavy data engineering (ETL) phase via spark which performs in-memory processing and is quite fast than traditional pandas for big data.

a. Data Extraction Tool:

- The extraction layer for our pipeline is built on Google Colab acting as the orchestration client, integrating directly with Google Drive as the raw data lake.
- We have stored our datasets on Google drive and we mount our drive on colab environment to access those datasets.
- The core extraction is handled by Apache Spark 3.5.1
- Spark is configured to handle complex, non-standard CSV formats commonly found in metadata_heavy files.
- The extraction logic specifically utilizes Spark's read options:
  .option("quote", "").option("escape", "").option("multiLine", True)
  to correctly parse CSVs where track names contain nested quotes or line breaks (like classical music movements or tracks with detailed feature credits).
- OpenJDK 11 and Spark binaries have been set up and installed in the Colab environment.

b. Data Processing Tool:

- We are using the processing engine, PySpark (Python API for Apache Spark).
- We selected this to handle the computational complexity of joining our spotify dataset with millions of records and for performing window aggregations that would be inefficient in standard Python.
- The pre-processing logic is explained in description of data processing.
- It mainly consists of following phases:
  - Cleaning and Normalization - text processing and filtering
  - Entity Resolution - algorithmic matching and manual mapping
  - Stratification and Ranking - window function and filtering
  - Storage Optimization - using parquet formats for saving reproducible datasets

c. Data Visualization:

- Our visualizations are interactive, built using Pandas, Seaborn and ipywidgets
- The datasets are loaded using pandas, the Parquet files are loaded into memory, enabling low-latency interaction

- The interface uses ipywidgets, widgets.Dropdown and widgets.interact to create a user-controlled environment.
- Users can dynamically switch the statistical aggregation method (Mean vs. Median) and specific audio feature being analyzed (e.g., Valence, Danceability, Tempo etc.)
- We have shown a comparative line plot, which is a dynamic Seaborn line chart compares the three cohorts (Critically Acclaimed, Listener Hits, General Population) over the 23-year period.
- Then we also show a horizontal stacked bar chart, that visualizes the "Taste Gap" in genres, showing the proportion of top genres across the three different groups.

Section 9: Challenges and lesson learned

a. Challenges Encountered:

1. Problem: The raw Spotify dataset consisted text features which were not structured and contained characters apart from just alphabets. This led to complex names of songs which although being the same track_name were different from standard and actual track names, listed in Grammy dataset (e.g., Classical music with movements, or Hip-Hop tracks with "feat." credits). This caused standard CSV parser to "shift" columns, resulting in numeric columns (like year) containing text data.

   Solution: We had to move beyond standard parsing and implement a custom PySpark reader configuration to correctly interpret the complex text structures without breaking our schema.

2. Problem: Merging the Grammy dataset with Spotify was not straightforward. Minor inconsistencies in naming conventions caused join failures. (Example: A track might be labeled "Crazy in Love" in one dataset and "Crazy in Love (feat. Jay-Z)" in the other or "Beyoncé" (with accent) vs. "Beyonce" (without)).

   Solution: We implemented Regex cleaning to strip text inside parenthesis and normalized the strings to lowercase before mapping. Even after cleaning, specific records failed to match. We manually mapped the index of datasets after checking 54 unmatched records to ensure the analysis was accurate.

3. Problem: Comparing the three groups presented a statistical challenge regarding variance.
   - Grammy Group: N = ~5 winner per year (High Volatility)
   - Popular Group: N = 100 songs per year
   - General Group: N = Thousands of songs per year

   Impact: In our line visualization, the Grammy line appeared extremely volatile as compared to the smooth "general population" line, making direct comparison a bit tricky.

b. Lessons Learned:

1. Even though our final dataset was small, the sourced dataset was massive to deal with. Using PySpark was essential not just for the size of dataset, but also

for its window functions. Trying to rank a million of songs to find the top N songs per year, using standard python/pandas would have been computationally inefficient. Distributed computing provided a good performance for sorting and ranking.

2. We learned that the CSV is a poor format for intermediate storage because it loses datatype information (confusing with the quotes). Hence, we converted our reproducible datasets to Parquet which preserved the schema and datatypes, allowing for a seamless transition from the data engineering environment to the visualization environment without type-casting errors.

3. Regex can fix discrepancies up to certain extent but for highly complex texts, it may fail. Since we were dealing with prestigious specific data points, we wanted to be accurate with our information and hence we learned that a hybrid approach of automating 90% of the matching and then manually mapping the remaining 10% unmatched data is often a more pragmatic solution for attaining high precision.

4. We also learnt that to get a better insight from visualization, it is better to plot datasets of comparable sizes, otherwise the plot may appear volatile for one and smooth for other dataset.

Section 10: References

[1] Apache Spark. (2016). Apache Spark 3.5.1 Documentation. https://archive.apache.org/dist/spark/docs/3.5.1

[2] Apache Spark. (2016). Testing PySpark. https://archive.apache.org/dist/spark/docs/3.5.1/api/python/getting_started/testing_pyspark.html

[3] JohnPendenque. (n.d.). Grammy Winners and Nominees from 1965 to 2024 [Data set]. Kaggle. https://www.kaggle.com/datasets/johnpendenque/grammy-winners-and-nominees-from-1965-to-2024

[4] AmitanshJoshi. (n.d.). Spotify 1 Million Tracks [Data set]. Kaggle. https://www.kaggle.com/datasets/amitanshjoshi/spotify-1million-tracks

[5] Spotify. (n.d.). Spotify for Developers: Web API. https://developer.spotify.com/documentation/web-api

[6] Formatting issues when reading a CSV file using PySpark's spark.read.csv function. Stack Overflow. https://stackoverflow.com/questions/75811760/formatting-issues-when-reading-a-csv-file-using-pysparks-spark-read-csv-functio

[7] Evergreen, S., & Emery, A. K. (2016). Data Visualization Checklist (May 2016 edition). https://stephanieevergreen.com/wp-content/uploads/2016/10/DataVizChecklist_May2016.pdf